FRI

Priprava in oblikovanje zaključnega dela

Peter Klepec

Magistrski študijski program Računalništvo in informatika, smer Računalništvo in informatika

Fakulteta za računalništvo in informatiko

Magistrsko delo

Mentor: izr. prof. dr. Lan Kabel
Somentor: prof. dr. Samo Bajt
Somentor: prof. dr. Primer Drugega Somentorja

Copyright: To delo je ponujeno pod licenco CC-BY-4.0. Razvita programska oprema in rezultati dela so na voljo v okviru GPL-3.0-or-later na https://git.fri.uni-lj.si/UL-FRI/thesis.

V tem zaključnem delu obravnavamo praktično uporabo in prednosti stavčnega sistema LaTeX pri pripravi tega konkretnega zaključnega dela. Pisanje obsežnejšega strokovnega besedila na Fakulteti za računalništvo in informatiko (FRI) prinaša številne izzive na področju tipografije, doslednosti oblikovanja in urejanja bibliografije. Tradicionalni vizualni urejevalniki besedil pogosto povzročajo neenakomerne razmike in težave pri poravnavi, medtem ko LaTeX z logičnim pristopom zagotavlja visoko stopnjo uniformnosti, konsistentnosti in prenosljivosti tega dokumenta. V delu podrobno predstavljamo strukturo, ki jo uporablja to zaključno delo, slogovne smernice ter pravilno vključevanje in sklicevanje na ključne elemente, kot so plovke (slike in tabele), programski izseki ter psevdokoda. Poseben poudarek namenimo uveljavljenim slovenskim tipografskim pravilom in pravilnemu zapisu matematičnih konstruktov in matrik. Prav tako demonstriramo integracijo sistema BibLaTeX za avtomatizirano in standardizirano upravljanje ter citiranje uporabljene literature. To besedilo služi kot neposreden praktični vodnik in tehnična predloga, ki študentom omogoča učinkovito strukturiranje lastnih misli ter doseganje stroge skladnosti in z zahtevanimi akademskimi standardi.

Zaključno delo | LaTeX | Tipografija | Oblikovanje besedil | Tehnično pisanje

Preparation and Formatting of a Final Thesis

This final thesis explores the practical application and advantages of the LaTeX typesetting system used directly in the preparation of this specific work. Writing a comprehensive technical text at the Faculty of Computer and Information Science (FRI) presents numerous challenges regarding typography, formatting consistency, and bibliography management. Traditional visual word processors frequently introduce uneven spacing and alignment issues, whereas LaTeX utilizes a logical approach to text processing that ensures a high degree of document uniformity, consistency, and portability across this entire document. The thesis provides a detailed overview of the structure implemented in this final thesis, stylistic guidelines, and the correct integration and referencing of key elements, including floats (figures and tables), code snippets, and pseudocode. Special attention is dedicated to established Slovenian typographical rules and the proper notation of mathematical constructs and matrices. Furthermore, we demonstrate the integration of the BibLaTeX system for the automated, standardized management and citation of the referenced literature. The presented text serves as a direct practical guide and a technical template, enabling students to efficiently structure their research and achieve strict compliance with academic standards.

Final Thesis | LaTeX | Typography | Text Formatting | Technical Writing

Prvi koristen nasvet v zvezi z uporabo LaTeXa pri pisanju vašega zaključnega dela je, da v celoti preberete ta dokument!

Datoteka na kratko opisuje, kako se pisanja zaključnega dela lotimo z uporabo programskega okolja LaTeX [1, 2].

LaTeX omogoča logično urejanje besedil, ki ima v primerjavi z vizualnim urejanjem številne prednosti, saj se problema urejanja besedil loti s programerskega stališča. Logično urejanje besedil omogoča večjo konsistentnost, uniformnost in prenosljivost besedil, predvsem pa nam omogoča lomljenje besed in posledično lepo obojestransko poravnavo. Obojestranska poravnava v vizualnih urejevalnikih povzroči kupe odvečnih in neenakomernih presledkov.

Razdelek 1 na kratko predstavi tipične sestavne dele zaključnega dela in slogovne smernice. V 2. razdelku spoznamo osnovne gradnike LaTeXa, nasvete za pisanje, uporabo količin in enot ter pravila za sklice in citiranje. V 3. razdelku opozarjamo na najpogostejše slovnične in tipografske napake, ki jih delamo v slovenščini. Razdelek 4 predstavi vključevanje plovk: vstavljanje in formatiranje posameznih slik in tabel ter postavljanje več elementov hkrati. V 5. razdelku spoznamo vključevanje programske kode in algoritmov. V 6. razdelku so predstavljeni matematično okolje, zapis matrik in sistemov enačb, besedilni konstrukti (kot so izreki in dokazi) ter sklicevanje nanje. V 7. razdelku se bomo srečali z iskanjem virov in sklicevanjem na literaturo z uporabo BibLaTeXa. Na koncu sledijo še sklepne ugotovitve z opomnikom za preverjanje pred oddajo naloge.

Ta vzorec ni priročnik za uporabo LaTeXa, saj razloži le nekatere osnovne ukaze, druge funkcionalnosti pa le omeni. Kako se jih uporablja, pa naj bralec poišče drugje.

1. Struktura zaključnega dela

Zaključno delo ima sicer predvideno okvirno strukturo, dejanska struktura pa je precej odvisna od področja, v katerem delujete, zato se glede tega nujno posvetujte z mentorjem. Ustaljena struktura sicer bralcu, predvsem pa mentorju in članom komisije, omogoča hitro navigacijo in preverjanje znanstvene integritete opravljenega dela.

Tipična struktura zaključnega dela na FRI obsega obvezne in priporočene gradnike, ki sledijo spodaj. Predvidena dolžina zaključnih del v tej predlogi, vključno z referencami, vendar brez morebitnih dodatkov, je:

- 8 do 12 strani za diplome in
- 11 do 20 strani za magistrske.

Pri tem za oboje velja:

- priporočena dolžina povzetka tako v slovenščini kot v angleščini od 150 do 250 besed,
- priporočena dolžina daljšega slovenskega povzetka (na koncu zaključnega dela, vendar pred morebitnimi dodatki) je 10 % dolžine celotnega dela (brez dodatkov),
- je podanih 4 do 6 ključnih besed.

Okvirni gradniki zaključnega dela:

Naslovna stran poleg naslova v slovenščini in angleščini vsebuje še ime avtorja, mentorja, morebitnega somentorja ter letnico in kraj izdaje. Naslov mora biti jedrnat, a hkrati dovolj informativen, da bralec takoj prepozna domeno računalništva, ki jo naloga obravnava.

Povzetek (slovenski in angleški) je kratek, samostojen tekst (običajno omejen na približno 250 besed), ki povzame motivacijo, uporabljene metode in najpomembnejše rezultate. Na FRI je obvezen tudi prevod povzetka v angleščino (*Abstract*), kar povečuje vidnost dela v mednarodnem okolju.

Ključne besede so izbrani termini, ki se uporabljajo za indeksiranje naloge v digitalnih knjižnicah. Izbrati velja besede, ki specifično opisujejo tehnologije, algoritme ali področja (npr. strojno učenje, ogrodje React, generativni modeli).

Uvodni razdelek pripravi teren za razumevanje problema. V njem jasno opredelimo stanje področja, motivacijo za delo, cilje naloge in hipoteze, ki jih želimo preveriti. Uvod se običajno zaključi s kratko predstavitvijo vsebine po posameznih razdelkih, kar bralcu služi kot zemljevid. Pozorni bodite, da ne pišete števil razdelkov ročno, ampak jih referencirate z `\ref{xx}` in poskrbite, da imate pri naslovih razdelkov `\label{xx}`.

Pregled področja in sorodnih del je namenjen umeščanju naloge v obstoječ kontekst. Tukaj predstavite katere rešitve za izbrani problem že obstajajo in v čem se vaša rešitev razlikuje od tistih, ki so jih razvili drugi avtorji. Temeljit pregled literature pokaže vašo razgledanost na področju.

Osrednji razdelki tvorijo hrbtenico naloge. Pri računalniških zaključnih delih tipično zajemajo sledeče razdelke, vendar pa jih lahko razbijate na več ločenih razdelkov in podrazdelkov, jih poimenujete po svoje ali odstranite, povsem odvisno od področja v katerem delujete. O tem se predvsem posvetujte z mentorjem:

- **Metodologija oz. teoretične osnove:** Razlaga konceptov, potrebnih za razumevanje rešitve.
- **Analiza in načrtovanje:** Opis arhitekture rešitve, omejitev, zahtev ipd.
- **Implementacija:** Opis rešitve, podrobnosti razvoja, uporabljene knjižnice in reševanje specifičnih tehničnih izzivov, vendar brez dejanskih številk.
- **Eksperimenti, evalvacija in rezultati:** Testiranje rešitve, primerjava zmogljivosti ali uporabniška študija, predstavitev rezultatov in argumentacija. Tipično se to razbije na več razdelkov, odvisno od količine izvedenih eksperimentov.

Zaključek povzame dosežene cilje in poda kritično presojo opravljenega dela. Pomembno je, da izpostavite omejitve svoje rešitve in nakažete smeri za morebitno nadaljnje delo ali izboljšave.

Literatura in viri je seznam vseh strokovnih in znanstvenih člankov, knjig in spletnih virov, na katere se sklicujete. Vsak vir v seznamu mora imeti ustrezen sklic v besedilu.

Priloge so namenjene gradivu, ki bi preveč motilo tok besedila v osrednjem delu, a je pomembno za popolno razumevanje (npr. obsežni kosi programske kode, vprašalniki ali podrobni diagrami).

1.1. Slogovne smernice in priporočila.

Zavedajte se, da vrstni red branja ne sledi vrstnemu redu pisanja zaključnega dela. Verjetno se boste lotili pisanja sorodnih del in metodologije pred eksperimenti ali rezultati, vsekakor pa pred uvodnim razdelkom ali povzetkom, ki ga zagotovo napišite povsem na koncu. Tega se zavedajo tudi bralci, zato se vam pri pisanju uvoda ni potrebno pretvarjati, da eksperimenti še niso bili opravljeni.

- Strokovna besedila morajo biti nedvoumna, zato naj bodo stavki kratki, preprosti in razumljivi, terminologija dosledna.
- Pri pisanju zaključnega dela upoštevajte akademski slog, ki se močno razlikuje od poljudnega pisanja. Besedilo pišite v prvi osebi množine (npr. »razvili smo«, »opazimo«), saj s tem poudarite objektivnost raziskovalnega procesa, v katerega sta vključena študent in mentor. Dovoljena je tudi uporaba trpnika, ki pa je jezikovno okornejša (»razvit je bil«, »izdelano je bilo«). Prve osebe ednine (»razvil sem«, »izdelal sem«) ne uporabljajte oziroma jo uporabite izjemoma samo tam, kjer želite poudariti svojo osebno odločitev med več možnostmi.

- Vse trditve v nalogi morajo biti preverljive. To pomeni, da morajo temeljiti na lastnih izpeljavah ali testiranjih, če pa povzemate ugotovitve drugih, je navajanje vira obvezno.
- Za posamezen pojem ali objekt uporabljajte v celotnem delu enako ime in oznako (če je potrebna).
- Če vpeljujete nove pojme, pazite, da so nedvoumno in natančno definirani.
- Uporabljajte uveljavljeno strokovno terminologijo.
- S pisanjem strokovnih besedil se razvija tudi slovenska strokovna terminologija. Pri strokovnih izrazih, ki še nimajo prevoda v slovenščino, po potrebi in v sodelovanju z mentorjem poiščite ustrezen slovenski prevod.
- Posebna pozornost naj velja prevodom iz angleščine. Pazite na slovenska slovnična pravila. Samostalniški prilastki so v slovenščini na desni strani osebka ali predmeta (SQL query → poizvedba SQL, API interface → vmesnik API, x-axis → os x), obratni vrstni red, kot je na primer SQL poizvedba, Windows strežnik ali ACME sistem, je napačen.

2. Osnovni gradniki LaTeXa in nasveti za pisanje

LaTeX bi lahko najbolj preprosto opisali kot programski jezik namenjen oblikovanju besedil. Tako kot vsak visokonivojski programski jezik ima tudi LaTeX številne ukaze za oblikovanje besedila in okolja, ki omogočajo strukturiranje besedila.

Vsi LaTeXovi ukazi se začnejo z levo poševnico `\\`, okolja pa definiramo bodisi s parom zavitih oklepajev in ali z ukazoma `\begin{ }` in `\end{ }`. Ukazi imajo lahko tudi argumente, obvezni argumenti so podani v zavitih oklepajih, opcijski argumenti pa v oglatih oklepajih.

Z ukazi torej definiramo naslov in imena avtorjev besedila, razdelke in podrazdelke in po potrebi bolj podrobno strukturiramo besedila na spiske, navedke itd. Posebna okolja so namenjena zapisu matematičnih izrazov, kratki primeri so v naslednjem razdelku.

Vse besedilne konstrukte lahko poimenujemo in se s pomočjo teh imen nato kjerkoli v besedilu nanje tudi sklicujemo.

LaTeX sam razporeja besede v odstavke tako, da optimizira razmike med besedami v celotnem odstavku. Nov odstavek začnemo tako, da izpustimo v izvornem besedilu prazno vrstico. Da besedilo skoči v novo vrstico, pa ukažemo z dvema levima poševnicama. Število presledkov med besedami v izvornem besedilu ni pomembno.

2.1. Koristni nasveti pri pisanju v LaTeXu. Programski paket LaTeX je bil prvotno predstavljen v priročniku [1] in je v resnici nadgradnja sistema TeX avtorja Donalda Knutha [3], znanega po svojih knjigah o umetnosti programiranja ter Knuth-Bendixovem algoritmu [4]. TeX in njegove izpeljanke so odprtokodni programi.

Različnih implementacij LaTeXa je cela vrsta, a priporočamo spletno verzijo, ki poenostavi sodelovanje pri pisanju, Overleaf.

Včasih smo si pri pisanju v LaTeXu pomagali predvsem s tiskanimi priročniki [1], danes pa je enostavneje in hitreje, da ob vsakem problemu za pomoč enostavno povprašamo najljubši jezikovni model.

LaTeX včasih ne zna pravilno deliti slovenskih besed, ki vsebujejo črke s strešicami. Če taka beseda štrli preko desnega roba, lahko LaTeXu pokažemo, kje se tako besedo deli takole: `ra\-ču\-nal\-ni\-štvo`. Katere vrstice so predolge, lahko vidimo tako, da dokument prevedemo s vključeno opcijo `draft`: `\documentclass[a4paper, 12pt, draft]{book}`.

V izvornem besedilu lahko začenjate vsak stavek v novi vrstici, saj LaTeX sam razporeja besede po vrsticah postavljenega besedila. Na ta način je tudi iskanje po izvornem besedilu in popravljanje veliko hitrejše. Večina sistemov za TeXiranje sicer omogoča s klikanjem enostavno prestopanje iz prevedenega besedila na ustrezno mesto v izvornem besedilu in obratno.

Boljšo preglednost dosežemo tako kot pri pisanju programske kode – z vizualnim urejanjem kode in izpuščanjem praznih vrstic. Pri spreminjanju in dodajanju izvornega besedila je najbolje pogosto prevajati, da se sproti prepričamo, če so naši nameni pravilno izpolnjeni.

Kadar besedilo, ki je že bilo napisano z nekim vizualnim urejevalnikom (npr. z Wordom), želimo prenesti v LaTeX, je tudi najbolje to delati postopoma s posameznimi bloki besedila, tako da lahko morebitne napake hitro identificiramo in odpravimo. Za prevajanje Wordovih datotek v LaTeX – in obratno – sicer obstajajo prevajalniki, ki pa običajno ne generirajo tako čisto logične strukture besedila, kot jo sicer LaTeX omogoča. Hiter in enostaven način prevedbe besedila, ki zahteva sicer ročne dopolnitve, lahko

poteka tudi tako, da besedilo urejeno z vizualnim urejevalnikom najprej shranimo v formatu pdf, nato pa to besedilo uvozimo v urejevalnik, kjer urejamo izvorno besedilo v formatu LaTeX.

2.2. Pisave v LaTeXu. V LaTeXovem okolju lahko načeloma uporabljamo poljubne pisave. Izbira poljubne pisave pa ni tako enostavna kot v vizualnih urejevalnikih besedil. Posamezne oblikovno medseboj usklajene pisave so običajno združene v družine pisav. V LaTeXu se privzeta družina pisav imenuje Computer Modern, kjer so poleg navadnih črk (roman v LaTeXu) na voljo tudi kurzivne črke (*italic* v LaTeXu), krepke (**bold** v LaTeXu), kapitelke (SMALL CAPS v LaTeXu), linearne črke (sans serif v LaTeXu), pisava pisalnega stroja (`typewriter` v LaTeXu) in nekatere njihove kombinacije, npr. krepke linearne črke (**sans serif** v LaTeXu). V istem dokumentu zaradi skladnega izleda uporabljamo običajno le pisave ene družine. Pomembna je tudi konsistentna raba več pisav in da ne pretiravamo z mešanjem različnih pisav.

Ko začenjamo uporabljati LaTeX, je zato najbolj smiselno uporabljati kar privzete pisave, s katerimi je napisan tudi ta dokument. Z ustreznimi ukazi lahko nato preklapljamo med navadnimi, kurzivnimi, krepkimi in drugimi pisavami. Zelo enostavna je tudi izbira velikosti črk. LaTeX odlično podpira večjezičnost, tudi v sklopu istega dokumenta, saj obstajajo pisave za praktično vse jezike, tudi take, ki ne uporabljajo latinskih črk.

2.3. Skladnost s standardom PDF/A. Repozitorij Univerze v Ljubljani zahteva, da so oddana zaključna dela skladna s standardom PDF/A. V predlogi smo zagotovili skladnost s standardom PDF/A-2b. S predlogo žal ne moremo preprečiti, da bi uporabnik v diplomsko delo dodal elemente, ki niso skladni s standardom. Pred oddajo zaključnega dela zato uporabnikom priporočamo preverjanje skladnosti z ustreznimi preverjevalniki, npr. z orodjem veraPDF[1].

2.4. Uporaba količin in enot. Pri tehničnem in strokovnem pisanju je zelo pomembno dosledno razlikovanje med količinami in njihovimi pripadajočimi merskimi enotami. Neustrezen zapis lahko vodi do napačne interpretacije podatkov in zmanjša strokovno vrednost dokumenta.

Količina predstavlja lastnost sistema, strojne ali programske opreme, ki jo je mogoče kvantitativno določiti. Simbole količin (npr. C za kapaciteto pomnilnika, f za frekvenco procesorja, B za pasovno širino) pišemo praviloma v **ležeči pisavi**.

Merska enota je dogovorjena vrednost, s katero primerjamo isto vrsto količine. Simbole enot (npr. B, Hz, $\mathrm{bit\,s^{-1}}$) pišemo v **pokončni pisavi**.

Med številčno vrednostjo in pripadajočo enoto mora biti vedno prisoten **stični presledek** (tilda), tako kot pri citatih. To zagotavlja, da vrednost in enota ob morebitnem prelomu vrstice ostaneta skupaj.

Za dosego standardiziranega zapisa v okolju LaTeX uporabljamo paket `siunitx`. Ta omogoča avtomatsko upravljanje s presledki in decimalnimi ločili. Spodaj so navedeni primeri pravilne uporabe:

- **Izpis vrednosti z enoto:** Ukav `\qty{100}{\giga\byte}` vrne 100 GB.
- **Posebni znaki:** Odstotke in stopinje pišemo s presledkom (za to že poskrbi LaTeX), npr.:
  - `\qty{98}{\percent}` vrne 98 %,
  - `\qty{85}{\degreeCelsius}` vrne 85 °C.

V slovenščini kot decimalno ločilo uporabljamo vejico, kar v paketu `siunitx` nastavimo globalno z ukazom `output-decimal-marker = {,}`.

Še primer rabe enot v besedilu: Pri obremenitvenem testu strežnika smo ugotovili, da pri frekvenci $f = 3.5\,\mathrm{GHz}$ in obremenitvi 98 % prepustnost omrežja doseže $B = 1.2\,\mathrm{Gbit\,s^{-1}}$. Vse meritve so bile opravljene pri temperaturi procesorja $T = 85\,^{\circ}\mathrm{C} \pm 1.5\,^{\circ}\mathrm{C}$.

2.4.1. Računalnikarji, spoštujte SI. Pri pisanju se zavedajte, da cel svet, z izjemo Liberije, Mjanmarja in Združenih držav Amerike, uradno uporablja Mednarodni sistem enot (SI), ki nam vsakodnevno lajša življenje. Ali ni čudovito reči, da ste letos prevozili 23 Mm, namesto nepoučenega dolgovezenja s 23 000 km?

Pri zapisovanju velikosti v bajtih prihaja zaradi zgodovinskih razlogov do pogostih težav. Velja sledeče:

[1] https://verapdf.org/

- Če ste uporabili desetiški pretvornik (1000), potem uporabite SI predpone: k (kilo), M (mega), G (giga), T (tera), tj. kB, MB, GB, TB.
- Če ste uporabili dvojiški pretvornik (1024), potem uporabite dvojiške predpone: kibi (Ki), mebi (Mi), gibi (Gi), tebi (Ti), tj. KiB, MiB, GiB, TiB. Naj vas ne zmoti dejstvo, da se veliko proizvajalcev in celo določeni operacijski sistemi tega ne držijo (lahko da prihajajo iz katere od treh posebnih držav).

Seveda ne pozabite tudi na razliko v zapisu bajta B in bita bit!

2.5. Sklici in citiranje. Pri pisanju strokovnih besedil je nujno dosledno sklicevanje na slike, tabele, enačbe in druge razdelke ter citiranje uporabljene literature. Sistem LaTeX nam to močno olajša, saj ob prevajanju samodejno poskrbi za pravilno oštevilčenje. Uporabljamo naslednje osnovne ukaze:

- **Označevanje (`\label`):** Element, na katerega se želimo sklicujemo, moramo najprej označiti z ukazom `\label{ime_oznake}`. Priporočljivo je, da za boljšo preglednost uporabljate predpone, na primer `fig:` za slike, `tab:` za tabele in `sec:` za razdelke (npr. `\label{sec:osnove}`).
- **Sklicevanje (`\ref`):** V besedilu se na označeni element skličemo z ukazom `~\ref{ime_oznake}`. LaTeX bo na tem mestu izpisal ustrezno številko. Če se skličemo na matematično enačbo, namesto tega uporabimo ukaz `~\eqref{ime_oznake}`, ki številko enačbe samodejno obda z okroglimi oklepaji.
- **Citiranje (`\cite`):** Za navajanje virov iz bibliografske datoteke uporabimo ukaz `~\cite{kljuc_vira}`.

Zelo pomembno pravilo: Med besedo in samim sklicem ali citatom mora vedno stati **nedeljivi presledek**, ki ga v LaTeXu označimo s tildo (~). S tem preprečimo, da bi številka sklica ob prelomu vrstice padla sama v novo vrstico, kar je tipografsko nepravilno.

Pravilna raba v besedilu torej izgleda takole: Kot lahko vidimo na sliki `~\ref{sl:example_coral_reef}`, velja enačba`~\eqref{en:masa}`. Podobno velja za literaturo: Algoritem je opisan v razdelku`~\cite{knuth}`.

Za navajanje neoštevilčenih poglavij in podpoglavij lahko uporabimo ukaz `\nameref`, ki nam izpiše ime poglavja oziroma podpoglavja, npr. v razdelku Plovke: slike in tabele ali podrazdelku Formati slik.

2.6. Seznami. Predloga podpira več okolij, ki ustvarijo sezname. Delimo jih na redka (`itemize` in `enumerate`) ter zgoščena okolja (`compactitemize` in `compactenumerate`). Redki neurejeni in urejeni seznami so primerni, kadar želimo našteti več primerov z daljšimi opisi. Neurejeni seznam je videti takole. Dolge opise v neurejenih seznamih smo lahko videli v nekaterih izmed ostalih vzorčnih primerov, zato jih tu ne bomo ponavljali. Prikažimo le uporabo zgoščenega neurejenega in urejenega seznama. Smiselno je opozoriti, da je omogočena tudi uporaba dodatnih argumentov, tako kot v običajnem okolju `itemize` in `enumerate`.

Podajamo primer neurejenega seznama:

- prvi pogoj,
- drugi pogoj,
- tretji pogoj,

in primer urejenega seznama:

a) prvi predpogoj,
b) drugi predpogoj.

3. Pogoste napake pri pisanju v slovenščini

Pri pisanju strokovnih besedil v slovenščini se pogosto srečujemo s tipičnimi napakami, ki izvirajo iz vpliva tujih jezikov ali nepoznavanja tipografskih pravil LaTeXu. Spodaj so zbrana ključna pravila, organizirana po kategorijah:

- **Vrstni red besed in kratic:** Pridevniki, ki se ne sklanjajo (npr. kratice), morajo v slovenščini stati za samostalnikom. Pravilno pišemo »model CAD« in **ne** »CAD model«.
- **Sklanjanje tujih imen:** Pri sklanjanju tujih lastnih imen v slovenščini ne uporabljamo vezajev. Pravilno je »Applov operacijski sistem« in **ne** »Apple-ov«.

- **Ločila in stičnost:** Pika, klicaj in vprašaj so levostični (pred njimi ni presledka, za njimi pa je). Klicajev in vprašajev se v strokovnih besedilih načeloma izogibamo. Oklepaji so desnostični, zaklepaji pa levostični: »(takole)«.

- **Tropičje in naštevanje:** Tropičja enostavno v zaključnih delih ne uporabljajte. V znanstvenih besedilih in zaključnih delih uporabljamo itd. ali idr. in ne tropičja. Pozorni bodite, da lahko uporabite le npr. ali itd. in ne obeh v eni povedi.

  Za kasnejšo rabo tropičja v življenju pa vendarle velja sledeče. V slovenščini je tropičje praviloma nestično ločilo. V LaTeXu ga zapišemo z ukazom `\dots` in ne s tremi pikami. Primer: »pes, maček, rosomah, … « Izjema je tropičje, ki mu sledi drugo ločilo; takrat je levonestično, npr. »pes, maček, rosomah, …!«

- **Narekovaji:** So stična ločila. V LaTeXu ne smemo uporabiti zgolj tipke `"` ali dveh `''`, saj se začetni in končni narekovaji ne bodo pravilno izrisali. Najboljša praksa je uporaba ukaza `\enquote{...}`, ki samodejno postavi pravilne slovenske narekovaje: »spodaj devetice, zgoraj šestice«.

- **Vezaj in pomišljaj:**

  - **Vezaj (-):** Je levo in desno stičen, npr. `slovensko-angleški` in označuje enakovredno razmerje med pojmoma. V kodi ga pišemo z enim znakom `-`.

  - **Pomišljaj (–):** V slovenščini in v (britanski) angleščini je nestičen (s presledki). V kodi ga zapišemo z dvema znakoma `\text{Pozor -- hud pes!}`

  - **Razponski pomišljaj (–):** Uporabljamo ga za razpone (npr. »od–do«), je stičen: `strani 7--11` in se ga v kodi piše povsem enako kot pomišljaj (`--`).

  - **Ameriški pomišljaj (—):** t.i. *em-dash* je daljši, stičen in se v kodi piše s tremi znaki `---`, opravlja pa povsem enako vlogo kot nestični pomišljaj v slovenščini oz. v britanski angleščini t.i. *en-dash*, npr. `Beware---vicious dog!` V slovenščini ga seveda ne uporabljajte.

- **Presledki ob ločilih in kraticah:** LaTeX privzeto uporablja t.i. francoski razmik, kar pomeni, da je presledek po koncu stavka malenkost večji od tistega med besedami. To vodi do dveh specifičnih težav:

  - **Kratice sredi stavka:** Če piki za kratico (npr. *prof.*, *dr.*, *itd.*) sledi presledek, bo LaTeX to interpretiral kot konec stavka in vstavil predolg razmik. To preprečimo z uporabo tilde `\~` (stični presledek) ali poševnice s presledkom. Pravilno: `prof.~dr.~Novak` ali `npr.\ v tem primeru`.

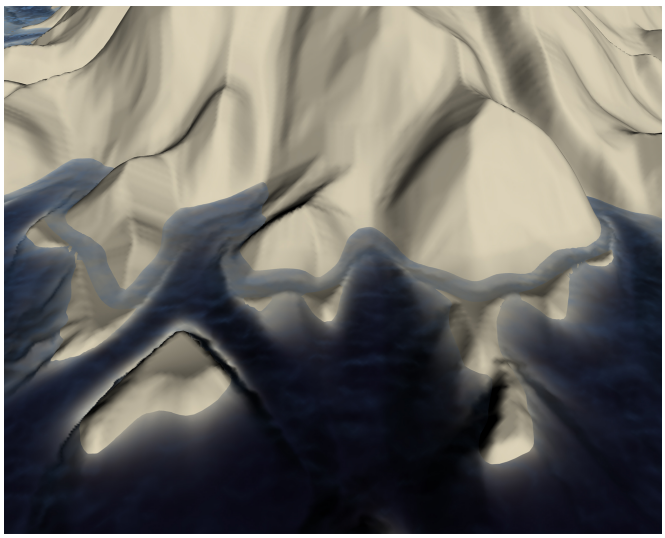
  - **Kratice na koncu stavka:** Če se stavek konča z veliko črko (npr. kratica na koncu stavka), LaTeX misli, da gre za kratico sredi stavka in vstavi premajhen razmik. Za pravilno interpretacijo konca stavka pred piko vstavimo ukaz `\@`. Pravilno: `Razvili so novo CPE\@. Sledila je testna faza.`

- **Vejice in stavčna zgradba:** Vejica loči stavke (povedke), razen če so zvezani z vezniki *in, pa, ter, ne–ne, niti–niti, ali, bodisi, oziroma.* Čeprav velja pravilo »pred ki, ko, ker, da, če, vejica skače«, je potrebno paziti na vrivke in večbesedne veznike. V strokovnih besedilih je bolje pisati krajše in jasne stavke. Veliki jezikovni modeli vam bodo pri postavljanju vejic v veliko pomoč, za zares pravilno postavljanje pa svetujemo uporabo spletnega orodja CJVT vejice (https://orodja.cjvt.si/vejice/).

- **Terminologija in viri:** Za nove računalniške pojme si pomagamo z uveljavljenimi slovarji. Glavni viri so *Islovar* Slovenskega društva Informatika [5], Slovar Slovenskega društva za razpoznavanje vzorcev [6] ter osrednje jezikovno spletišče *Fran* [7].

4. Plovke: slike in tabele

Slike in daljše tabele praviloma vključujemo v dokument kot plovke. Pozicija plovke v končnem izdelku ni pogojena s tekom besedila, temveč z izgledom strani. LaTeX bo skušal plovko postaviti samostojno, praviloma na mestu, kjer se pojavi v izvornem besedilu, sicer pa na vrhu strani, na kateri se na takšno plovko prvič sklicujemo. Pri tem pa bo na vsako stran končnega izdelka želel postaviti tudi sorazmerno velik del besedila. V skrajnem primeru, če imamo res preveč plovk na enem mestu besedila, ali če je plovka previsoka, se bo LaTeX odločil za stran popolnoma zapolnjeno s plovkami.



Slika 1. Virtualno obogatena skulptura [8]. Rezultate računalniško generirane animacije z video projektorjem projeciramo na kamnito skulpturo, da ustvarimo vtis, kot da bi po skulpturi polzele vodne kapljice [9, 10].

Poleg tega, da na položaj plovke vplivamo s tem, kam jo umestimo v izvorno besedilo, lahko na položaj plovke na posamezni strani prevedenega besedila dodatno vplivamo z opcijami `here, top` in `bottom`, vendar se temu izogibajte, če je le možno. Zelo velike slike je najbolje postaviti na posebno stran z opcijo `page`. Skaliranje slik po njihovi širini lahko prilagodimo širini strani tako, da kot enoto za širino uporabimo kar širino strani, npr. `0.5\textwidth` bo raztegnilo sliko na polovico širine strani. Sliko lahko po potrebi tudi zavrtimo za 90 stopinj in jo razstegnemo na višino strani. Tako bodo podrobnosti na sliki lažje berljive in prostor na strani bo bolje izkoriščen.

Na vse plovke se moramo v besedilu sklicevati, saj kot beseda plovka pove, plovke plovejo po besedilu in se ne pojavijo točno tam, kjer nastopajo v izvornem besedilu. Vendar naj bosta sklic na plovko v besedilu in sama plovka v oblikovanem besedilu čim bližje skupaj, tako da bralcu ne bo potrebno listati po zaključnem delu. Upoštevajte pa, da se naloge tiska dvostransko in da se hkrati vidi dve strani v dokumentu! Na to, kje se bo slika ali druga plovka pojavila v postavljenem besedilu lahko torej najbolj vplivamo tako, da v izvorni kodi plovko premikamo po besedilu nazaj ali naprej!

Tabele je najbolje oblikovati kar neposredno v LaTeXu, saj za oblikovanje tabel obstaja zelo fleksibilno okolje `tabular`. Slike pa so po drugi strani pogosto najlažje oblikovati oziroma izdelati z drugimi orodji in programi, rezultate shraniti v formatu `.pdf` ali `.jpeg` in nato v LaTeXu le vključiti ustrezno slikovno datoteko. Za pisanje besed, ki so vključene v slike, uporabite pisave/fonte, ki so čimbolj podobne pisavam v samem besedilu, ali še bolje shranite brez fonta in pustite LaTeXu, da vključi svojega.

Knjižnica https://en.wikibooks.org/wiki/LaTeX/PGF/TikZ pa omogoča risanje raznovrstnih grafov neposredno v okolju LaTeX.

Na vse tabele in slike se moramo v besedilu sklicujemo, saj kot plovke v oblikovanem besedilu niso nujno na istem mestu kot v izvornem besedilu. Pri sklicevanju na slike uporabimo veliko začetnico, npr. "glej Sliko 1", saj gre za ime slike.

4.1. Formati slik. V dokument LaTeX lahko vključimo slike različnih formatov, tako rasterske slike kot vektorske slike.

Za grafe in diagrame so najbolj primerne vektorske slike v formatu `.pdf`. Ne samo, da bo besedilo predstavljeno kot besedilo, ampak bodo slike lepe in berljive tudi ob veliki povečavi. Še več, shranjevanje v vektorski obliki od vas običajno ne zahteva nič dodatnega truda. Če npr. graf narišete v Pythonu, le poskrbite, da ga shranite kot `.pdf` in ne kot npr. `.png`, nikakor pa ne kot `.jpg`.

Če vključujete fotografije oz. nekaj česar niste narisali pa bo to verjetno shranjeno v rasterski obliki in shranjeno kot `.jpg` ali `.png`. Slika 1 je npr. v formatu `.jpg`, vendar na povsem enak način vključite tudi npr. `pdf`.

4.2. Podnapisi k slikam in tabelam. Vsaki sliki ali tabeli moramo dodati podnapis, ki nam pojasnjuje, kaj je na sliki ali tabeli, obrazloži vse morebitne okrajšave, pojasni glavna dognanja. Ne bojte se daljših opisov. Če nekdo le prelista zaključno delo, naj bi že iz slik in njihovih podnapisov lahko na grobo razbral, kakšno temo naloga obravnava.

Če slike povzamemo iz drugih virov, se moramo v podnapisu k taki sliki nujno sklicujemo na ta vir!

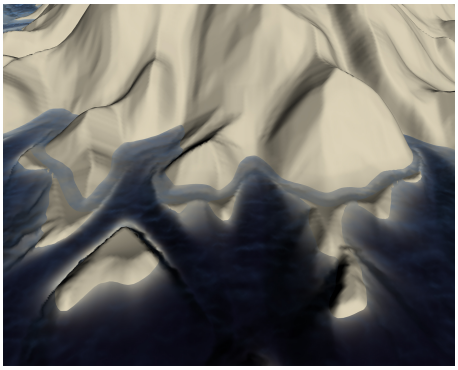
4.3. Vključevanje več slik in tabel hkrati. Pri pisanju zaključnega dela se pogosto pojavi potreba po prikazovanju več slik ali tabel ene ob drugi. Za ta namen uporabljamo paket `subcaption`, ki predstavlja sodoben in podprt standard v LaTeXu.

V spletnih virih in starih predlogah študenti prepogosto najdejo in uporabijo paketa `subfigure` ali `subfig`. Oba paketa sta zastarela in pogosto povzročata težave pri prevajanju ali konflikte z drugimi paketi (npr. pri generiranju hiperpovezav). Teh dveh paketov zato **ne uporabljajte**. Prav tako se za razporejanje slik v mrežo ne poslužujte paketa `TikZ`. Ta paket je sicer izjemno močno orodje, a je namenjen risanju vektorskih shem in je za preprosto razporejanje slik odločno preveč kompleksen.

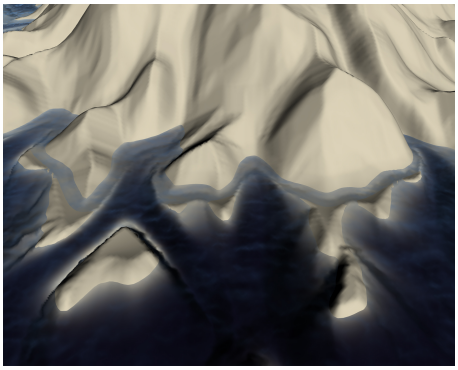
Za uporabo sodobnega pristopa v preambulo dokumenta dodajte ukaz: `\usepackage{subcaption}`. Paket nam ponuja dve okolji: `subfigure` za slike in `subtable` za tabele. Obe okolji v ozadju delujeta na podobnem principu kot standardno okolje `minipage` (ki ga v LaTeXu pogosto uporabljamo za postavljanje povsem neodvisnih blokov vsebin enega ob drugega), saj obema podamo eksplicitno širino, ki jo bosta zavzela.

Ker so slike in tabele plavajoči objekti, ne smemo nikoli predpostavljati, kje natančno na strani se bodo izrisale. Na vse elemente se v besedilu vedno sklicujemo izključno z ukazom `\ref`, pred katerim uporabimo nedeljivi presledek (`~`).

4.3.1. Več slik s podnaslovi. Kadar želimo vsaki sliki dodeliti svojo oznako (npr. a, b) in podnaslov, uporabimo okolje `subfigure`. Širino posamezne podslike določimo z ulomkom širine besedila, npr. `0.48\textwidth`. Vmesni prostor do polne širine zapolnimo z ukazom `\hfill`. Na sliki 2 lahko vidimo primer dveh slik, ki sta postavljeni ena ob drugi.



(a) Podnaslov leve slike.



(b) Podnaslov desne slike.

Slika 2. Skupni naslov, ki opisuje obe sliki.

Za prehod v novo vrsto znotraj istega plavajočega objekta preprosto izpustimo prazno vrstico in po želji dodamo nekaj vertikalnega prostora z ukazom `\vspace`. Na tak način je zgrajena mreža štirih slik na sliki 3.

4.3.2. Več slik brez podnaslovov (mreža brez presledkov). Če individualni podnaslovi niso potrebni in želimo le en skupen naslov celotne mreže slik, okolja `subfigure` sploh ne potrebujemo. Slike preprosto naložimo znotraj navadnega `figure` okolja in določimo njihovo velikost.

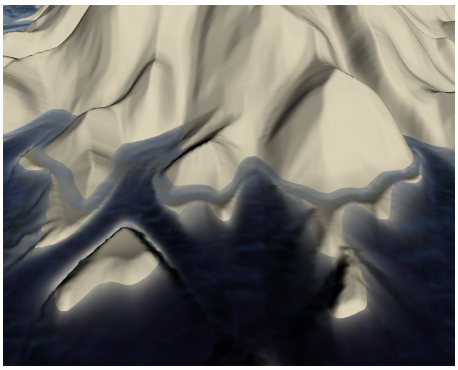
Če želimo ustvariti neprekinjeno mrežo povsem **brez vmesnih presledkov** (kot to prikazuje slika 4), moramo poskrbeti za dve stvari:

1. **Horizontalni presledki:** Širino določimo tako, da je vsota vseh širin v vrstici točno enaka `1.0\textwidth` (npr. štiri slike s širino `0.25\textwidth`). Na konec vsake vrstice z ukazom `\includegraphics` moramo dodati znak za komentar (`%`), s čimer preprečimo, da bi LaTeX prelom vrstice interpretiral kot presledek.

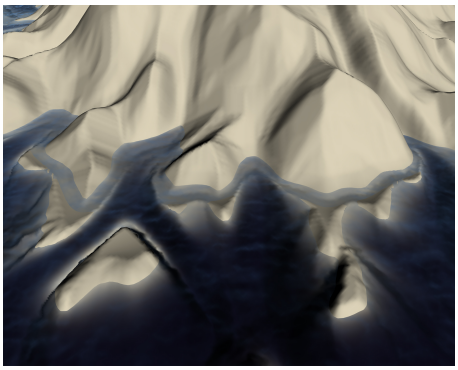
2. **Vertikalni presledki:** Znotraj posebnega bloka moramo uporabiti ukaz `\offinterlineskip`, ki v celoti izklopi samodejno vrivanje vertikalnega prostora med vrsticami. Pomembno je, da blok po zadnji sliki zapremo z }, preden definiramo `\caption`.

Pri sliki 4 je vključena je tudi širša postavitev z uporabo bloka `{largefigure}`, namesto `{figure}`.

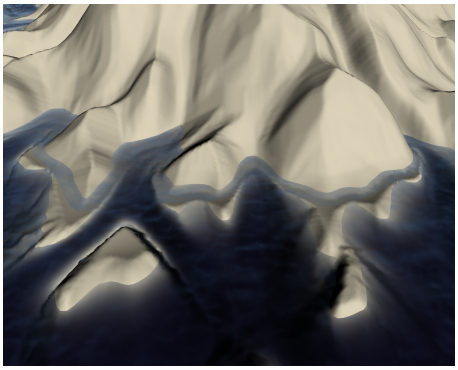
4.3.3. Več tabel ena ob drugi. Logika postavljanja dveh tabel ene ob drugo je popolnoma enaka kot pri slikah, le da znotraj okolja `table` uporabimo okolje `subtable`. Edino pomembno pravilo, na katerega ne smete pozabiti, je: **podnaslovi tabel vedno sodijo nad tabelo**, medtem ko podnaslovi slik sodijo pod njo.



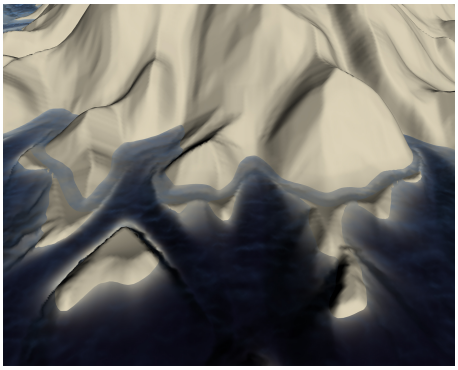
(a) Zgoraj levo.



(b) Zgoraj desno.

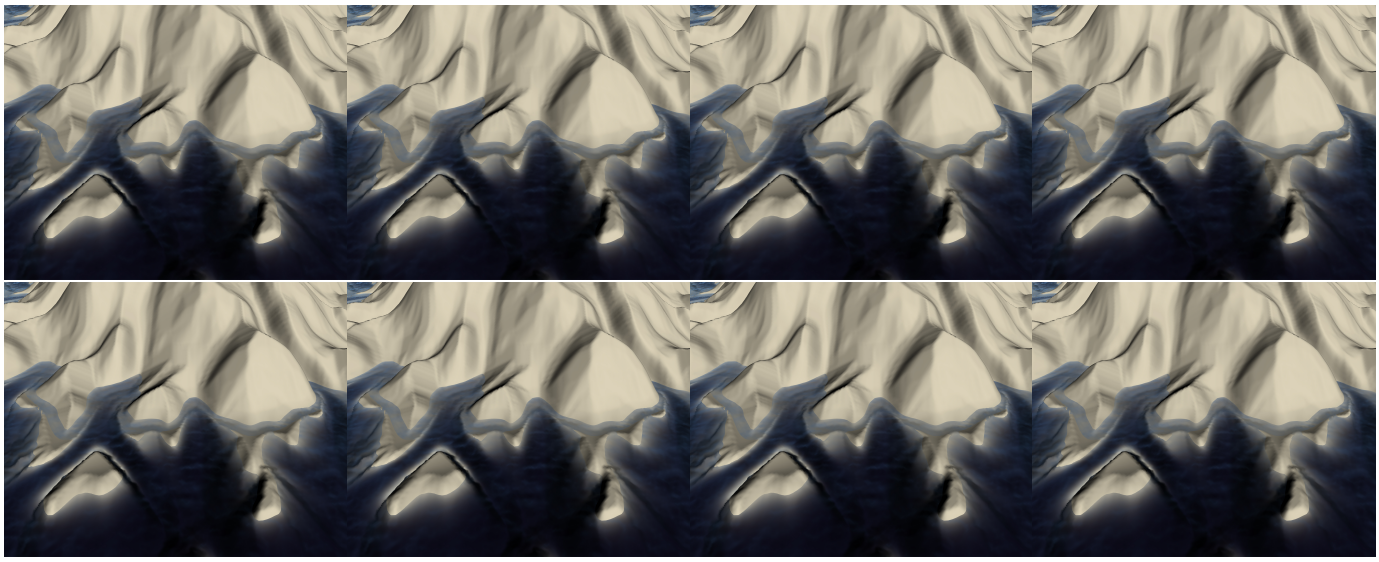


(c) Spodaj levo.



(d) Spodaj desno.

Slika 3. Mreža štirih slik s posameznimi podnaslovi.



Slika 4. Mreža osmih slik brez vmesnih presledkov in z le enim skupnim naslovom. Vključena je tudi širša postavitev.

Če ugotovite, da je vsebina tabele preširoka in povzroča prekrivanje, lahko celotno okolje `tabular` ovijete v ukaz `\resizebox{\ linewidth}{!}{...}`, ki tabelo proporcionalno pomanjša, da se natančno prilega širini podtabele. Tabela 1 prikazuje ustrezen zapis na primeru vrednotenja modelov računalniškega vida.

Tabela 1. Primerjava uspešnosti klasifikacije slik z različnimi arhitekturami globokih nevronskih mrež na dveh standardnih podatkovnih zbirkah. Pri metriki *točnost* (angl. *accuracy*) so boljše višje vrednosti (označeno s puščico ↑), pri metriki *napaka* (angl. *error rate*) pa so boljše nižje vrednosti (označeno s puščico ↓).

(a) Rezultati na zbirki CIFAR-10.

arhitektura	točnost (↑)	napaka (↓)
ResNet-18	93,5 %	6,5 %
ResNet-50	95,2 %	4,8 %
ViT-Base	97,8 %	2,2 %

(b) Rezultati na zbirki ImageNet.

arhitektura	točnost (↑)	napaka (↓)
ResNet-18	69,7 %	30,3 %
ResNet-50	76,1 %	23,9 %
ViT-Base	84,2 %	15,8 %

Omogočena je tudi uporaba širokih tabel, kot vidimo iz tabele 2.

5. Vključevanje programske kode in algoritmov

Svojo programsko kodo v poljubnem programskem jeziku najlažje vključimo s paketom `minted`. Okoli bloka `\minted{ }` sicer lahko podamo klasični `\listing{ }` in skupaj z dodanim `\caption` dobimo vse potrebno za prikaz kode, vendar pa se takšen blok ne razteza pravilno skozi več strani. Ena od možnosti za rešitev je uporaba bloka `\center{ }`, za napis pa `\captionof{ }`, kot je prikazano pri obeh primerih izsekov kode 1 in 2.

Izsek kode 1. Kratek primer kode v programskem jeziku Python.

```python
def python_veverica():
    # Oh, kako je to lepo!
    x = -1
    print("Res izgleda čudovito.")
```

Kar je nadvse čudovito pri paketu `minted` je to, da lahko podamo dele kode znotraj vrstice, kot je npr. tole: `python_veverica()`.

Izsek kode 2. Malo daljši primer programske kode v jeziku Javascript, ki se lahko razteza skozi dve strani.

```javascript
async function pridobiUporabnika(id) {
    const url = `https://api.primer.fri/users/${id}`;

    try {
        const odgovor = await fetch(url);

        // Preverimo, če je statusni kod v območju 200-299
        if (!odgovor.ok) {
            console.error(`Napaka pri povezavi: ${odgovor.status}`);
            throw new Error(`Uporabnika z ID ${id} ni bilo mogoče najti.`);
        }

        const podatki = await odgovor.json();

        // Logiranje uspešnega prejema podatkov
        console.log("Podatki so bili uspešno prejeti.");
        console.log(`Ime uporabnika: ${podatki.name}`);

        return podatki;

    } catch (napaka) {
        // Tukaj ulovimo tako mrežne napake kot napake pri branju JSON-a
        console.warn("Prišlo je do nepričakovane težave.");
        return { error: napaka.message };
    }
}
```

Vodoravni črti pri začetku in koncu izseka, v kombinaciji s sivim ozadjem, vizualno sporočata morebitno raztezanje skozi več strani. To je razvidno pri izseku kode 2.

5.1. Vključevanje algoritmov (psevdokode). Medtem ko paket `minted` uporabljamo za prikaz dejanske implementacije v specifičnem programskem jeziku, v zaključnih delih pogosto opisujemo samo logiko reševanja problema na višjem nivoju. Za zapis algoritmov v

Tabela 2. Primer široke tabele. Prikazani so časi izvajanja treh namišljenih algoritmov. Razvidno je, da ima algoritem A eksponentno časovno zahtevnost, algoritem B kvadratno časovno zahtevnost in algoritem C linearno časovno zahtevnost. Tovrstne podatke je sicer bolj smiselno prikazovati na grafu, a za demonstracijo uporabe široke tabele bi moralo zadoščati.

n	čas izvajanja algoritma A	čas izvajanja algoritma B	čas izvajanja algoritma C
5	2 s	3 s	4 s
10	61 s	9 s	6 s
15	33 min	18 s	8 s
20	17 h	32 s	10 s

obliki psevdokode je standardna izbira kombinacija paketov `algorithm` in `algpseudocode` (slednji je del paketa `algorithmicx`).

Za razliko od izsekov izvorne kode, algoritme običajno obravnavamo kot standardne plavajoče objekte (tako kot slike in tabele). To pomeni, da LaTeX sam poišče optimalno mesto za njihov prikaz na strani. V besedilu se nanje sklicujemo z nedeljivim presledkom in ukazom `\ref`, na primer: Algoritem 1 prikazuje klasičen postopek za iskanje največjega skupnega delitelja.

Naslov (`\caption`) pri algoritmih običajno postavljamo na vrh, tik pod začetek okolja `algorithm`. Številka `[1]` ob začetku bloka `algortihmic` pa LaTeXu sporoči, naj oštevilči vsako vrstico algoritma, kar nam olajša sklicevanje na posamezne korake v besedilu.

Algoritem 1. Evklidov algoritem za iskanje največjega skupnega delitelja

Require: $a \geq 0 \vee b \geq 0$
Ensure: Največji skupni delitelj a in b

```
1: procedure EVKLID(a, b)
2:     while b ≠ 0 do
3:         t ← b
4:         b ← a mod b
5:         a ← t
6:     return a
```

6. Matematično okolje in sklicevanje na besedilne konstrukte

Poglejmo nekaj primerov pisanja enačb, sklicevanj nanje, podajanja dokazov, matrik, razbijanje enačb in nekaj splošnih tipografskih pravil v matematiki.

Izrek 1. *Za vsako naravno število n velja*

$$n < 2^n. \tag{1}$$

Dokaz. Dokazovanje z indukcijo zahteva, da neenakost (1) najprej preverimo za najmanjše naravno število – 0. Res, ker je $0 < 1 = 2^0$, je neenačba (1) za $n = 0$ izpolnjena.

Enačbo (1) smo referencirali z `~\eqref{eq:1}` in ne z `~\ref{eq:1}`, saj nam slednji ne ovije reference v oklepaje. Brez oklepajev bi bralec lahko referenco na enačbo zamenjal z referenco na razdelek, z oglatimi oklepaji pa na sklic v literaturi.

Za potrebe primera nadaljujmo z indukcijskim korakom. S predpostavko, da je neenakost (1) veljavna pri nekem naravnem številu n, je potrebno pokazati, da je ista neenakost v veljavi tudi pri njegovem nasledniku – naravnem številu $n + 1$. Računajmo.

$$\begin{aligned} n + 1 &< 2^n + 1 && (2)\\ &\leq 2^n + 2^n && (3)\\ &= 2^{n+1} \end{aligned}$$

Neenakost (2) je posledica indukcijske predpostavke, neenakost (3) pa enostavno dejstvo, da je za vsako naravno število n izraz 2^n vsaj tako velik kot 1. S tem je dokaz Izreka 1 zaključen. □

Ste opazili kako so indukcijski koraki lepo poravnani? Uporabili smo pomemben `align`. Ste opazili, da smo stavek pri neenakosti 1 končali s piko? V primeru, da je enačba del povedi, kot npr.

$$y = 2^x, \tag{4}$$

pa seveda dodamo vejico v samo enačbo, kot je prikazno pri (4).

Za kakovostno tehnično pisanje pa moramo obvladati še zapis matrik in obdelavo zelo dolgih enačb.

6.1. Matrike. Za zapis matrik uporabljamo okolja iz paketa `amsmath`. Najpogosteje uporabljamo okolje `bmatrix` za matrike z oglatimi oklepaji ali `pmatrix` za matrike z okroglimi oklepaji.

Primer zapisa matrike A:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}. \tag{5}$$

Pri tem elemente v vrstici ločimo z znakom `\&`, vrstice pa zaključimo z `\\`. Za različne vrste pik uporabljamo ukaze `\dots` (vodoravne), `\vdots` (navpične) in `\ddots` (diagonalne).

6.2. Razbijanje dolgih enačb. Kadar je enačba predolga za eno vrstico, uporabimo okolje `split` znotraj okolja `equation`. To nam omogoča, da enačbo razbijemo na several vrstic, medtem ko celoten blok obdrži le eno skupno številko.

$$\begin{aligned} f(x) &= (a+b)^4 \\ &= a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4. \end{aligned} \tag{6}$$

Znak `\&` določa točko poravnave (običajno pri enačaju), `\\` pa prehod v novo vrstico.

6.3. Sistemi enačb in primeri. Za zapis funkcij, ki so definirane po delih, ali za sisteme enačb z enim velikim oklepajem uporabljamo okolje `cases`:

$$|x| = \begin{cases} x, & \text{če je } x \geq 0, \\ -x, & \text{če je } x < 0. \end{cases} \tag{7}$$

Opazimo uporabo ukaza `\text{...}`, ki omogoča vstavljanje običajnega besedila (z vejicami in presledki) neposredno v matematično okolje, kar ohranja pravilno pokončno pisavo.

6.4. Splošna tipografska pravila v matematiki in matematična okolja. Pri pisanju matematike upoštevamo naslednja pravila:

- **Vejice in pike:** Enačbe so del povedi, zato morajo vsebovati ločila. Če se poved konča z enačbo, na koncu enačbe stoji pika.
- **Standardne funkcije:** Funkcije, kot so $\sin, \cos, \ln, \max$, pišemo s pokončno pisavo z uporabo ustreznih ukazov (npr. `\sin`), ne pa kot *sin* (v ležeči pisavi), kar bi nakazovalo produkt spremenljivk s, i in n.
- **Presledki:** LaTeX samodejno upravlja s presledki v matematiki. Ročno poseganje običajno ni potrebno, razen v redkih primerih, kjer uporabimo npr. `\,` za majhen premik.

Avtomatsko se prevajajo tudi naslednja okolja za zapisovanje matematičnih izrekov.

Definicija 2. Problem Π je v razredu NP, če velja ...

Aksiom 1 (Komutativnost Boolove algebre)**.** Če je $A, \vee, \wedge$ Boolova algebra. Za vsak par $x, y \in A$ velja:

$$x \vee y = y \vee x$$

in

$$x \wedge y = y \wedge x.$$

Trditev 3. *Naj bo Σ NP-poln problem. Če obstaja Karpova prevedba problem Σ na problem Π, potem je Π NP-težek.*

Opomba 4. Obstoj Karpove prevedbe problema Π na problem Σ ne zagotavlja NP-težkosti problema Π.

Izrek 5. *Naj velja*

Dokaz. ... □

Posledica 6. ...

Lema 7. ...

Zgled 8. ... ♢

7. Kaj pa literatura?

Kot smo omenili že v uvodu, je pravi način za citiranje literature uporaba BibLaTeXa [11]. BibLaTeX zagotovi, da pri določeni vrsti literature ne izpustimo nobene obvezne informacije in da vse informacije dosledno navajamo na enak način in po istem vrstnem redu. BibLaTeX je nadgradnja starejšega sistema BibLaTeX. Novejši sistem je bolje prilagojen slovenščini in navajanju spletnih virov. Sicer pa so starejše datoteke `.bib` kompatibilne z BibLaTeXom.

Osnovna ideja BibLaTeXa je, da vse informacije o literaturi zapisujemo v posebna datoteko, v našem primeru je to `literatura.bib`. Vsakemu viru v tej datoteki določimo simbolično ime. V našem primeru je v tej datoteki nekaj najbolj značilnih zvrsti literature, kot so knjige [1], članki v revijah [12] in zbornikih konferenc [13], poglavja v knjigah [14], spletni viri [9, 15], tehnično poročilo [16], zaključna dela [17] itd. Zaključno delo [17] iz leta 1990 je bila prva diploma na tedanji Fakulteti za elektrotehniko in računalništvo, ki je bila oblikovana z LaTeXom. Reference, ki so na spletnih straneh arhivirane v elektronski obliki, imajo običajno številko DOI (http://dx.doi.org), ki jo zato tudi vključimo v izpis literature in tako bralcu elektronske verzije naše publikacije ponudimo neposredno povezavo do elektronske kopije te reference.

Po vsaki spremembi pri sklicu na literaturo moramo najprej prevesti izvorno besedilo s prevajalnikom LaTeX, nato s prevajalnikom BibLaTeX, ki ustvari datoteko `vzorec_dip_Seminar.bbl`, in nato še dvakrat s prevajalnikom LaTeX. V okolju Overleaf je to večkratno prevajanje z različnimi prevajalniki uporabniku skrito. Zato tudi začetnim uporabnikom LaTeXa svetujemo uporabo Overleafa.

Kako se spisek literature nato izpiše (ali so posamezni viri razvrščeni po vrstnem redu sklicevanja, ali po abecedi priimkov prvih avtorjev, ali se imena avtorjev pišejo pred priimki itd.) je odvisno od parametrov paketa BibLaTeX. V zaključnem delu bomo uporabili parametre `style=numeric`, kar pomeni, da bodo sklici na literaturo v besedilu označeni z zaporednimi številkami, za vrstni red izpisa referenc pa `sorting=nty`, kar pomeni, da bodo reference urejene po priimkih prvih avtorjev, nato po naslovu reference in nazadnje po letu izdaje [18]. Zato je potrebno pri določenih zvrsteh literature, ki nima avtorjev, dodati parameter `key`, ki določi vrstni red vira po abecedi.

Ko začenjamo uporabljati BibLaTeX je lažje, če za urejanje datoteke `.bib` uporabljamo kar isti urejevalnik kot za urejanje datotek `.tex`, čeprav obstajajo tudi posebni urejevalniki oziroma programi za delo z datotekami `.bib`.

Le če se na določen vir v besedilu tudi sklicujemo, se bo ta vir pojavil tudi v spisku literature. Tako je avtomatično zagotovljeno, da se na vsak vir v seznamu literature tudi sklicujemo v besedilu zaključnega dela. V datoteki `.bib` imamo sicer lahko veliko več virov za literaturo, kot jih bomo uporabili v zaključnem delu.

7.1. Zbiranje virov za seznam literature.

Vire v formatu `.bib` lahko enostavno poiščemo in prekopiramo iz spletnih strani založnikov ali različnih akademskih spletnih portalov za iskanje znanstvene literature. Izvoz referenc v Google učenjaku še dodatno poenostavimo, če v nastavitvah izberemo BibLaTeX kot želeni format za izvoz navedb. Navedbe, ki jih prekopiramo iz Google učenjaka in drugih podobnih akademskih portalov, moramo pred uporabo nujno preveriti, saj so taki navedki pogosto generirani povsem avtomatično in lahko vsebujejo napačne ali nepopolne podatke. Najpogosteje je napačen tip publikacije! Pri velikih jezikovnih modelih pa je lahko celotna referenca izmišljena!

Pri sklicevanju na literaturo na koncu stavka moramo paziti, da je pika po ukazu `\cite{ }`. Da LaTeX ne bi delil vrstice ravno tako, da bi sklic na literaturo v oglatih oklepajih začel novo vrstico, moramo pred sklicem na literaturo dodati nedeljiv presledek (tilda): `~\cite{ }` in ne navadnega presledka!

Običajno se v besedilu sklicujemo na nek vir ali več virov na koncu trdilnega stavka. Kadar pa omenimo avtorja nekega vira, pa sklic običajno vstavimo za njegovim priimkom.

Dandanes se skoraj vsi pri iskanju informacij vedno najprej lotimo iskanja preko spleta ali velikih jezikovnih modelov. Rezultati takega iskanja pa so pogosto spletne strani, ki danes obstajajo, jutri pa jih morda ne bo več, ali pa vsaj ne v taki obliki, kot smo jo prebrali. Smisel navajanja literature pa je, da tudi po dolgih letih nekdo, ki bo bral vaše zaključno delo, lahko poišče vire, ki jih navajate v zaključnem delu.

Znanstveni rezultati, ki so objavljeni v obliki recenziranih člankov, bodisi v konferenčnih zbornikih, še bolje pa v znanstvenih revijah, so veliko bolj izčiščen in zanesljiv vir informacij, saj so taki članki šli skozi recenzijske postopke. Predvsem pa so taki članki stabilen vir informacij, saj se načeloma po njihovi objavi ne spreminjajo več. Skoraj vsi ti članki so dandanes dosegljivi tudi v elektronski obliki, bodisi v arhivih založnikov, univerzitetnih repozitorijih ali tudi na osebnih spletnih straneh njihovih avtorjev. Zato na spletu začenjamo iskati vire za strokovna besedila predvsem preko akademskih spletnih portalov, kot so npr. Google učenjak, Research Gate ali Academia, saj so na teh portalih rezultati iskanja le akademske publikacije. Pri velikih jezikovnih modelih bodite pozorni, saj so viri lahko izmišljeni.

Če je za dostop do nekega članka potrebno plačati, se obrnemo za pomoč in dodatne informacije na našo knjižnico.

Za označevanje člankov, ki so na voljo v elektronski obliki, se je v zadnjem času uveljavila oznaka DOI (https://www.doi.org), kar močno olajša iskanje teh referenc na spletu. Založniki tudi starejšim člankom, ki so na voljo v elektronski obliki, za nazaj določajo oznake DOI. Zato poskusite poiskati ustrezno oznako DOI za vsak članek, ki ga citirate in jo vključite v seznam literature.

Če res ne gre drugače, pa je pomembno, da pri sklicevanju na običajni spletni vir vedno navedemo tudi datum, kdaj smo dostopali do tega vira.

Z uporabo BibLaTeXa je možno natisniti seznam literature posebej za določene vrste referenc, na primer za članke v znanstvenih revijah, članke v konferenčnih zbornikih in poglavja v knjigah, kot je prikazano tudi v tem vzorcu zaključnega dela.

8. Sklepne ugotovitve

Uporaba LaTeXa in BibLaTeXa je pri zaključnem delu **obvezna**!

Preden zaključno delo oddate mentorjem, še enkrat preverite:

- pravilno rabo elementov (tako LaTeXgradnikov kot tudi ločil kratic, pomišljajev itd.),
- slovnico (lahko si pomagate s katerim od orodij),
- da ni nobeno besedilo prilepljeno direktno iz velikega jezikovnega modela (čisto vse morate prebrati in preveriti, če že ne popraviti),
- da je dovolj slikovnih elementov (preveč suhoparnega besedila ni za nikogar koristno),
- da je vsaka slika in tabela ne samo podnaslovljena, ampak tudi referencirana v besedilu.

Preden zaključno delo oddate na sistemu STUDIS, še enkrat preverite:

- če je izpisana pravilna smer študija, vaše ime, ime in naziv mentorja in somentorja,
- če so vse besede pravilno deljene in da ne segajo preko desnega roba (poravnavo po vrsticah lahko kontrolirate tako, da izvorno datoteko enkrat testno prevedete z opcijo `draft`, kar vam pokaže predolge vrstice),
- če ste vključili pravilne verzije slik, predvsem pri grafih in diagramih vektorske (`.pdf` ali `.eps`) in ne rasterskih (npr. `.jpg` ali `.png`),
- če ste naslovili vse napake in opozorila, ki se lahko pojavijo pri tvorjenju dokumenta (bodite pozorni predvsem na manjkajoče citate, ki jih lahko poiščete tudi z iskanjem vprašajev v končnem dokumentu).

ZAHVALA. Na tem mestu zapišite, komu se zahvaljujete za pomoč pri izdelavi diplomske naloge oziroma pri vašem študiju nasploh. Pazite, da ne boste koga pozabili. Utegnil vam bo zameriti. Temu se da izogniti tako, da celotno zahvalo izpustite.

Literatura

1. Lamport L (1986) *LaTEX: A Document Preparation System*. (Addison-Wesley).
2. Oetiker T, Partl H, Hyna I, Schlegl E (2006) Ne najkrajši uvod v LaTeX2ε.
3. Wikipedia contributors (2021) Donald knuth — Wikipedia, the free encyclopedia.
4. Knuth DE, Bendix PB (1983) Simple word problems in universal algebras in *Automation of Reasoning: Classical papers on computational logic 1957–1966*, eds. Siekmann JH, Wrightson G. (Springer), pp. 342–376.
5. Slovensko društvo Informatika (2021) Islovar.
6. Slovensko društvo za razpoznavanje vzorcev (2021) Slovar, Razpoznavanje vzorcev.
7. ZRC SAZU (2022) Slovarji Inštituta za slovenski jezik Frana Ramovša ZRC SAZU.
8. Solina F, Meden B (2017) Light fountain – a virtually enhanced stone sculpture. *Digital Creativity* 28(2):89–102.
9. Solina F (2015) Light fountain ii – galaxy.
10. Solina F (2021) *Skulpture/Sculptures 2012-2020, 2. izdaja / 2nd edition*. (Društvo likovnih umetnikov Ljubljana, Založba UL FRI, Ljubljana).
11. (1988) Bibliography management with BibLaTex.
12. Solina F (2004) 15 seconds of fame. *Leonardo* 37(2):105–110.
13. Ciuha P, Klemenc B, Solina F (2010) Visualization of concurrent tones in music with colours in *Proceedings of the 18th ACM international conference on Multimedia*. (ACM), pp. 1677–1680.
14. Peer P, Batagelj B (2009) Art—a perfect testbed for computer vision related research in *Recent Advances in Multimedia Signal Processing and Communications*. (Springer), pp. 611–629.
15. (year?) SLOVARJI.SI seznam slovenskih spletnih slovarjev.
16. Andersen MR et al. (2012) Kinect depth sensor evaluation for computer vision applications, (Department of Engineering, Aarhus University), Technical Report 6.
17. Balon A (1990) Vizualizacija, (Fakulteta za elektrotehniko in računalništvo, Univerza v Ljubljani), Diplomska naloga.
18. (year?) Biblatex – sophisticated bibliographies in latex.

Daljši slovenski povzetek

V primeru pisanja v angleščini morate pripraviti razširjeni slovenski povzetek, ki naj bo dolg predvidoma 10 % dolžine celotnega zaključnega dela (brez dodatkov), torej pri dolžini 10 strani naj bo ta povzetek dolg 1 stran.

Priloge

A. Primeri uporabe dodatka oz. prilog. V tem razdelku podajamo nekaj primerov uporabe prilog. Sem dajemo stvari, ki so za bralce uporabne, vendar so preveč obsežne, da bi jih dali v glavni del zaključnega dela, npr. dodatni eksperimenti oz. študije, dodatni slikovni primeri ipd. Primer dodatnih rezultatov je prikazan v tabeli 3.

Tabela 3. Izmišljen primer ovrednotenja zmogljivosti različnih arhitektur za razpoznavanje. Puščice v glavi tabele označujejo, ali višja (↑) oziroma nižja (↓) vrednost metrike pomeni boljšo zmogljivost. Najbolje ocenjeni model v posamezni kategoriji je poudarjen in označen. Povp. učin. označuje povprečno učinkovitost.

		Nenadzorovana množica (UERC)			Nadzorovana množica			Povp. učin.	
Arhitektura	**Param.**	**Točnost (%) ↑**	**AUC ↑**	**EER (%) ↓**	**Točnost (%) ↑**	**AUC ↑**	**EER (%) ↓**	**Zakasnitev [ms] ↓**	**Velikost [M] ↓**
ResNet-50	25,6M	88,4	0,921	5,12	95,1	0,970	2,10	12,4	4,2
EfficientNet-B4	19,3M	90,1	0,945	4,05	96,8	0,982	1,45	18,2	3,8
Vision Transformer	86,6M	92,5	0,961	3,20	97,5	0,988	1,10	24,8	8,5
GAN-Detect	45,1M	91,0	0,950	3,88	96,0	0,975	1,80	16,5	6,1
Mega-Base	32,4M	94,2	0,978	2,15	98,2	0,991	0,85	**10,2 ↓**	**3,5 ↓**
Mega-Large	95,8M	**96,8 ↑**	**0,989 ↑**	**1,05 ↓**	**99,1 ↑**	**0,998 ↑**	**0,42 ↓**	22,1	14,2
ResNet-50	25,6M	88,4	0,921	5,12	95,1	0,970	2,10	12,4	4,2
EfficientNet-B4	19,3M	90,1	0,945	4,05	96,8	0,982	1,45	18,2	3,8
Vision Transformer	86,6M	92,5	0,961	3,20	97,5	0,988	1,10	24,8	8,5
GAN-Detect	45,1M	91,0	0,950	3,88	96,0	0,975	1,80	16,5	6,1